PROJECT DOCUMENTATION

QueuePulse

Queue Monitoring and Reporting System

Student Name	Mohammed Kaif .S. Shaikh
AF Code	AF05315286
Project Type	Web Application
Technology Stack	PHP, Laravel, SQL, MySQL/MySQLi, HTML, CSS, JavaScript, XAMPP
Development Environment	XAMPP / Visual Studio Code
Document Version	1.0
Project Level	Beginner to Intermediate

Table of Contents

	Page No
1. Project Overview	3
2. Requirements	3
3. Technology Stack	4
4. System Architecture	5
5. Database Design	6
6. Module Description	8
7.Suggested Project Folder Structure	9
8. Development Methodology	12
9. Validation and Security	12
10. Testing Plan	13
11. Installation and Execution	13
12. Limitations	14
13. Future Enhancements	14
14. Conclusion	14
15. References	15

1. Project Overview

1.1 Project Title

QueuePulse – Queue Monitoring and Reporting System

1.2 Introduction

QueuePulse is a proposed web-based application intended to help people understand queue conditions at selected locations. Users can view location information and submit reports about queue waiting times. The system stores submitted reports, calculates an indicative waiting-time average, and presents a queue status to help users make more informed decisions.

1.3 Problem Statement

People often have limited information about how busy a service location is before visiting it. This can lead to unexpected waiting, wasted time, and poor planning. QueuePulse aims to provide a shared place for viewing recent queue reports and estimated waiting conditions.

1.4 Objectives

- Provide a centralized list of monitored locations.
- Allow users to submit queue waiting-time reports.
- Store and retrieve reports using a relational database.
- Calculate an indicative average from recent reports.
- Display a simple queue/crowd status for each location.
- Validate input and handle errors safely.
- Provide a maintainable structure for future enhancements.

1.5 Project Scope

The initial scope includes location listing, queue report submission, recent-report storage, average waiting-time calculation, and display of queue status. User accounts, administrator moderation, maps, notifications, analytics, and production deployment may be added later if required.

2. Requirements

2.1 Functional Requirements

- View monitored locations.
- View the current estimated queue status for a location.
- Submit a waiting-time report with a location and wait duration.
- Validate report fields before saving.

- Store reports with a timestamp.
- Calculate the average using a defined set of recent reports (planned: latest 20 reports per location).
- Update the location's displayed status after a new report.
- Display a friendly message when data is unavailable or invalid.

2.2 Non-Functional Requirements

- Usability: simple, readable, responsive interface.
- Reliability: handle database and validation failures gracefully.
- Performance: retrieve recent reports efficiently.
- Security: validate input, use safe database queries, and avoid exposing database credentials.
- Maintainability: keep application logic, views, and configuration organized.
- Compatibility: run locally using a supported PHP environment and XAMPP services.

2.3 Assumptions

QueuePulse reports are user-submitted and may not represent an official measurement. The displayed average and queue category are estimates. Exact queue thresholds should be agreed and documented before implementation.

3. Technology Stack

3.1 Technologies and Purpose

- HTML5 – structures pages and forms.
- CSS3 – styles the interface and responsive layout.
- JavaScript – client-side interaction and basic usability enhancements.
- PHP – server-side request handling and application logic.
- Laravel – routing, controllers, models, validation, configuration, and application organization.
- SQL – defines and queries relational data.
- MySQL – stores locations, reports, and calculated status data.
- MySQLi – PHP's MySQL interface for direct procedural/object-oriented database access where a standalone PHP module is used.
- XAMPP – local development environment providing Apache, PHP, and MySQL/MariaDB services.
- Visual Studio Code and a web browser – development and testing tools.

3.2 Important Database Connectivity Note

Laravel normally accesses MySQL through its database layer and configured PDO-based connection; Laravel applications generally should not open MySQLi connections directly

inside controllers or models. MySQLi is appropriate for a separate plain-PHP implementation or a specifically required standalone connection example. To avoid two competing database layers, the implementation should choose one consistent approach: Laravel's database layer for the Laravel application, or MySQLi for a separate plain-PHP version. If the course requires both Laravel and MySQLi, document clearly where each is used and do not mix them in the same data-access flow without a deliberate design.

4. System Architecture

4.1 Architecture Layers

Presentation Layer: HTML, CSS, JavaScript, and Laravel Blade templates render location cards, report forms, and status information.

Application Layer: Laravel routes and controllers receive requests, validate input, run queue calculations, and return responses.

Data Layer: Laravel models and migrations manage MySQL tables. A standalone plain-PHP variant may use MySQLi prepared statements if required.

4.2 Request Flow

1. A user opens QueuePulse in a browser.
2. The browser requests a page from the local web server.
3. Laravel routes the request to the appropriate controller.
4. The application validates submitted values.
5. The application reads or writes records in MySQL.
6. The system calculates the recent average and status where required.
7. A response is returned and displayed in the browser.

4.3 Suggested Laravel Structure

QueuePulse/

app/Http/Controllers/ (request handling)

app/Models/ (Location, QueueReport, QueueStatus)

database/migrations/ (database schema)

resources/views/ (Blade templates)

routes/web.php (web routes)

public/css/ (stylesheets)

public/js/ (JavaScript)

config/database.php (database configuration)

.env (local environment settings; do not commit secrets)

5. Database Design

5.1 Proposed Database

Database name: queuepulse

5.2 Table: locations

Stores the names and details of locations where users can report queue waiting times.

Column Name	Data Type	Description
id	INT	Primary key that uniquely identifies each location. Automatically increments for every new record.
name	VARCHAR(120)	Stores the location name, up to 120 characters.
address	VARCHAR(255)	Stores the optional address or description of the location, up to 255 characters.
created_at	TIMESTAMP	Stores the date and time when the location record was created.
updated_at	TIMESTAMP	Stores the date and time when the location record was last updated.

5.3 Table: queue_reports

Stores user-submitted reports of waiting times for different locations.

Column Name	Data Type	Description
id	BIGINT	Primary key that uniquely identifies each queue report. Automatically increments for every new record.
location_id	BIGINT	Foreign key referencing locations.id. Identifies the location associated with the report.
wait_minutes	INT UNSIGNED	Stores the reported waiting time in minutes. The value cannot be negative.
report_note	TEXT	Stores an optional note or additional information about the queue.
created_at	TIMESTAMP	Records when the queue report was submitted. Used to identify the most recent reports.
updated_at	TIMESTAMP	Records when the queue report was last modified.

5.4 Table: queue_status

Stores the calculated queue status and average waiting time for each location.

Column Name	Data Type	Description
id	BIGINT	Primary key that uniquely identifies each queue status record. Automatically increments for every new record.
location_id	BIGINT	Foreign key referencing locations.id. Identifies the location whose queue status is stored.
average_wait_minutes	DECIMAL(8,2)	Stores the calculated average waiting time in minutes, allowing two decimal places.
status_label	VARCHAR(30)	Stores the queue status, such as Low, Moderate, or High.
reports_count	INT UNSIGNED	Stores the number of reports used to calculate the average waiting time.
updated_at	TIMESTAMP	Records the date and time when the queue status was last calculated or updated.

5.5 Relationships and Rules

One location can have many queue reports. A location has one current calculated status in the proposed queue_status design. queue_reports.location_id and queue_status.location_id should reference locations.id. Use foreign keys and indexes where appropriate. If status can be calculated on demand, queue_status may be omitted and the value derived from queue_reports instead.

5.6 Example SQL Schema

```
CREATE DATABASE queuepulse;
USE queuepulse;

CREATE TABLE locations (
  id INT AUTO_INCREMENT PRIMARY KEY,
  name VARCHAR(120) NOT NULL,
  address VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP
);

CREATE TABLE queue_reports (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  location_id INT NOT NULL,
  wait_minutes INT UNSIGNED NOT NULL,
```

```
  report_note VARCHAR(255) NULL,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_reports_location
    FOREIGN KEY (location_id) REFERENCES locations(id),
  INDEX idx_reports_location_created (location_id, created_at)
);

CREATE TABLE queue_status (
  id BIGINT AUTO_INCREMENT PRIMARY KEY,
  location_id INT NOT NULL UNIQUE,
  average_wait_minutes DECIMAL(6,2) NOT NULL DEFAULT 0,
  status_label VARCHAR(20) NOT NULL DEFAULT 'Unknown',
  reports_count INT UNSIGNED NOT NULL DEFAULT 0,
  updated_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
    ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT fk_status_location
    FOREIGN KEY (location_id) REFERENCES locations(id)
);
```

6. Module Description

6.1 Location Listing

Retrieves locations from the database and displays them as cards or rows. Each location should have a stable identifier and a clear name.

6.2 Queue Report Submission

A user selects a location and enters a waiting time in minutes. The server checks that the location exists and that the waiting time is a valid non-negative or positive integer according to the chosen business rule, then stores the report.

6.3 Average Waiting-Time Calculation

The planned calculation selects the latest 20 reports for a location and computes their arithmetic mean. If fewer than 20 reports exist, the calculation uses the available reports. If no reports exist, the application should display 'No reports yet' rather than imply a measured average.

6.4 Queue Status Calculation

A status label can be derived from the calculated average. Example thresholds must be finalized by the project owner; any Low/Moderate/High thresholds used in the final application should be consistent across the code and documentation. These labels are estimates, not official crowd measurements.

6.5 Status Display

The homepage displays the location name, estimated average waiting time, status label, and an option to submit a report. The interface should show when the estimate was last updated if that information is available.

6.6 Validation and Error Handling

Reject missing location IDs, unknown locations, malformed waiting times, and values outside the allowed range. Show user-friendly errors and log technical details securely. Do not show database passwords or raw SQL errors in the browser.

7. Suggested Project Folder Structure

```
QueuePulse/
|
├── app/
|   ├── Http/
|   |   ├── Controllers/
|   |   |   ├── HomeController.php
|   |   |   ├── LocationController.php
|   |   |   └── QueueReportController.php
|   |   |
|   |   ├── Requests/
|   |   |   └── StoreQueueReportRequest.php
|   |   |
|   |   └── Middleware/
|   |
|   ├── Models/
|   |   ├── Location.php
|   |   ├── QueueReport.php
|   |   └── QueueStatus.php
```

```
|   |
|   └── Services/
|       └── QueueStatusService.php
|
├── bootstrap/
|
├── config/
|   └── database.php
|
├── database/
|   ├── migrations/
|   |   ├── create_locations_table.php
|   |   ├── create_queue_reports_table.php
|   |   └── create_queue_statuses_table.php
|   |
|   ├── seeders/
|   |   ├── DatabaseSeeder.php
|   |   └── LocationSeeder.php
|   |
|   └── factories/
|
├── public/
|   ├── index.php
|   ├── css/
|   |   └── style.css
|   ├── js/
|   |   └── script.js
|   └── images/
|
├── resources/
|   └── views/
|       ├── layouts/
|       |   └── app.blade.php
|       |
```

```
│       ├── home.blade.php
│       │
│       ├── locations/
│       │   └── index.blade.php
│       │
│       ├── reports/
│       │   ├── create.blade.php
│       │   └── success.blade.php
│       │
│       └── errors/
│
├── routes/
│   └── web.php
│
├── storage/
│
├── tests/
│   ├── Feature/
│   │   ├── HomePageTest.php
│   │   └── QueueReportTest.php
│   └── Unit/
│
├── .env
├── .env.example
├── .gitignore
├── artisan
├── composer.json
├── composer.lock
└── README.md
```

9. Development Methodology

8.1 Development Phases

1. Requirement analysis: identify target users and required features.
2. Design: plan routes, page layouts, database tables, and queue-status rules.
3. Environment setup: configure PHP, Composer, Laravel, XAMPP, and MySQL.
4. Database development: create migrations and seed sample locations.
5. Backend development: implement routes, controllers, models, and report calculations.
6. Frontend development: create Blade pages with HTML, CSS, and JavaScript.
7. Validation and security: validate all server-side inputs and protect state-changing requests.
8. Testing: test successful and invalid scenarios.
9. Documentation and deployment: record setup steps and known limitations.

8.2 Development Environment

Use a compatible PHP version and Laravel release, Composer, XAMPP, and a browser. Laravel version and PHP compatibility should be confirmed at setup time. The XAMPP database service may be MySQL or MariaDB depending on the installed package.

9. Validation and Security

9.1 Input Validation

- Location must exist in the locations table.
- Waiting time must be numeric and within the configured limits.
- Optional notes should have a maximum length and be safely displayed.
- Server-side validation is mandatory even when JavaScript validation is present.

9.2 Database Security

Use Laravel's query builder/Eloquent or parameterized prepared statements for database operations. For any standalone MySQLi module, use MySQLi prepared statements. Never concatenate untrusted values into SQL queries. Keep database credentials in local environment configuration and exclude .env from version control.

9.3 Web Security

Use Laravel CSRF protection for forms that change data. Escape user-supplied content in templates. Apply rate limiting or abuse controls if public report submission is enabled. Restrict administrative features if introduced. Do not claim authentication or administrator controls are implemented unless they are present in the actual application.

10. Testing Plan

10.1 Test Cases

Test ID	Test Scenario	Expected Result
TC01	Open homepage	The list of locations is displayed.
TC02	Open a location with no reports	The system displays a no-data message.
TC03	Submit a valid report	The report is successfully saved in the database.
TC04	Submit an empty waiting time	A validation error message is displayed.
TC05	Submit text or a negative waiting time	The system rejects the invalid input.
TC06	Submit a report for an unknown location	The request is rejected with an appropriate error message.
TC07	Add more than 20 reports	The average waiting time is calculated using only the latest 20 reports.
TC08	Add a new report	The displayed average waiting time and queue status are updated accordingly.
TC09	Database service unavailable	A controlled error message is displayed without exposing technical details.
TC10	Submit a form without a valid CSRF token	The request is rejected by Laravel's CSRF protection.

10.2 Types of Testing

Functional testing verifies the modules against requirements. Integration testing verifies Laravel/PHP database operations. Validation testing checks incorrect inputs. Security testing checks SQL injection protection, CSRF protection, output escaping, and secret handling. User acceptance testing confirms that the application is understandable and useful.

11. Installation and Execution

11.1 Prerequisites

XAMPP, a compatible PHP version, Composer, Laravel, MySQL/MariaDB, a web browser, and Visual Studio Code.

11.2 Planned Local Setup

1. Install and start the required XAMPP services, including Apache and the database service.
2. Create the Laravel project using Composer if it has not already been created.
3. Configure the database name, username, password, and host in the local .env file.
4. Create and run Laravel migrations.
5. Add sample locations using seeders or SQL.

6. Run the Laravel development server with `php artisan serve`, or configure Apache to point to Laravel's `public` directory.
7. Open the local URL shown by the development server and test each module.
8. Keep local credentials private and do not commit `.env` to GitHub.

11.3 Configuration Note

The exact commands and URLs depend on the project's Laravel version and local environment. For a Laravel project, the web root should be the `public` directory; exposing the project root directly through Apache is not recommended.

12. Limitations

Reports may be inaccurate, outdated, duplicated, or submitted in bad faith. QueuePulse provides estimates based on submitted data and is not a real-time sensor system unless additional integrations are implemented. The initial version may not include verified users, moderation, maps, notifications, or analytics.

13. Future Enhancements

- User accounts and report moderation.
- Location search, filtering, and map integration.
- Time-based trends and charts.
- Report expiry and weighting newer reports more heavily.
- Notifications when a queue becomes less busy.
- Administrative dashboard and abuse reporting.
- API integration with trusted data sources.
- Deployment to a production server with backups and monitoring.

14. Conclusion

QueuePulse is designed to demonstrate full-stack web development by combining a browser interface with server-side processing and relational data storage. Its main workflow is to display locations, collect waiting-time reports, calculate a recent average, and show an indicative queue status. The project can be expanded with authentication, moderation, mapping, analytics, and production deployment as requirements grow.

This document describes the proposed design and implementation plan. Before final submission, update it to match the actual code, routes, database schema, screenshots, test results, and features completed. Do not report a feature as completed until it has been implemented and tested.

15. References

- PHP Manual: https://www.php.net/docs.php
- Laravel Documentation: https://laravel.com/docs
- MySQL Documentation: https://dev.mysql.com/doc/
- MySQLi Manual: https://www.php.net/manual/en/book.mysqli.php
- MDN Web Docs: https://developer.mozilla.org/
- Apache Friends / XAMPP: https://www.apachefriends.org/
- OWASP: https://owasp.org/